

FUNDAÇÃO ESCOLA DE COMÉRCIO ÁLVARES PENTEADO
FECAP
CENTRO UNIVERSITÁRIO ÁLVARES PENTEADO ANÁLISE E
DESENVOLVIMENTO DE SISTEMAS

Ana Julia Bernardo Quintans - 26028857

Clayton Santos Ferreira – 26029419

Leonardo Zachello Encinas - 26028745

Riane Toscano - 26029235

Ricardo Branco Rios - 26029360

PROJETO INTEGRADO
ALGORITMO E LÓGICA DE PROGRAMAÇÃO
PROFESSOR(A) ORIENTADOR(A): LUCY MARI TABUTI

São Paulo

2026

Sumário

Sumário	2
1. INTRODUÇÃO	3
2. ESTRUTURA DE MODULARIZAÇÃO.....	4
2.1. Entrada e saída	5
3. VETORES	6
3.1. Entrada e Saída.....	6
4. MATRIZ	7
4.1. Entrada e saída	7

1. INTRODUÇÃO

O objetivo deste trabalho é demonstrar, na prática, o funcionamento das estruturas de modularização, vetores e matrizes, utilizando um sistema simples de validação de IP.

A estrutura de modularização foi utilizada para organizar o sistema em diferentes métodos, onde cada função possui uma responsabilidade específica, como validar IP, estilizar a interface, exibir logs e obter o endereço IP da máquina. Essa divisão facilita a organização, manutenção e reutilização do código.

A estrutura de vetores foi representada pela utilização da classe `List<string>`, responsável por armazenar os registros das tentativas de acesso realizadas no sistema. Nessa estrutura, são guardadas informações como data, horário, IP digitado e status da validação, permitindo que o programa exiba os últimos acessos registrados.

Já a estrutura de matrizes representa o armazenamento de dados em linhas e colunas. Embora o sistema desenvolvido não utilize matrizes diretamente, essa estrutura é importante na programação para organizar informações em formato de tabela, possibilitando o controle de múltiplos dados simultaneamente.

Essas estruturas são fundamentais porque fazem parte da base da lógica de programação e estão presentes em praticamente todos os sistemas computacionais. Compreender seu funcionamento permite desenvolver aplicações mais organizadas, eficientes e capazes de lidar com diferentes situações. Dessa forma, este trabalho contribui para o desenvolvimento do raciocínio lógico e para a construção de sistemas mais completos e funcionais.

2. ESTRUTURA DE MODULARIZAÇÃO

```
32 > private void AplicarTitleBarPreta()...
41
    1 referência
42 > private void AplicarEstiloTela()...
50
    3 referências
51 > private void EstilizarBotao(Button botao)...
59
    1 referência
60 > private void btIP_Click(object sender, EventArgs e)...
73
    2 referências
74 > private string ObterIP()...
86
    1 referência
87 > private void btvalidar_Click(object sender, EventArgs e)...
```

Essa parte do código representa uma estrutura de modularização, pois o programa foi dividido em vários métodos, onde cada um possui uma responsabilidade específica.

A modularização permite organizar melhor o sistema, facilitando a leitura, manutenção e reutilização do código.

No programa, existem métodos separados para estilizar a interface, obter o IP da máquina, validar o IP digitado e exibir os logs de acesso. Dessa forma, cada função executa apenas uma tarefa específica, evitando repetição de código e deixando o sistema mais organizado.

Por exemplo, o método ObterIP() é responsável apenas por localizar o endereço IP da máquina, enquanto o método EstilizarBotao() cuida apenas da aparência dos botões. Já os eventos dos botões (btIP_Click, btvalidar_Click e btlog_Click) controlam as ações executadas pelo usuário na interface.

Essa divisão em módulos melhora a manutenção do sistema, pois alterações podem ser feitas em uma função sem afetar o restante do programa.

2.1. Entrada e saída

Entrada → ações do usuário nos botões e IP digitado no campo de texto

Processamento → cada método executa uma função específica do sistema

Saída → exibição do IP, validação de acesso e apresentação dos logs na tela

3. VETORES

No código não existe vetor (array) diretamente

```
14 private List<string> logs = new List<string>();
```

Essa parte do código representa uma estrutura de armazenamento de dados semelhante a um vetor, utilizando a classe `List<string>`. Ela é responsável por armazenar várias informações de logs do sistema, como data, horário, endereço IP digitado e status de autorização.

Sempre que um usuário tenta validar um IP, uma nova informação é adicionada na lista através do comando `logs.Add()`. Posteriormente, o programa percorre e exibe os últimos registros armazenados.

A utilização dessa estrutura permite guardar múltiplos dados em sequência, facilitando o controle e histórico das tentativas de acesso realizadas no sistema.

3.1. Entrada e Saída

Entrada → IP digitado pelo usuário

Processamento → armazenamento das tentativas na lista `logs`

Saída → exibição dos últimos registros de acesso na tela

4. MATRIZ

No código atual não existe matriz.

A estrutura de matrizes é utilizada quando o programa precisa armazenar dados organizados em linhas e colunas. Diferente dos vetores ou listas, que armazenam informações em apenas uma dimensão, a matriz permite trabalhar com múltiplas dimensões de dados simultaneamente.

No sistema desenvolvido, não foi necessário utilizar matrizes, pois as informações de acesso e validação foram armazenadas utilizando listas (List<string>), que atendem melhor à necessidade do projeto. Porém, caso fosse necessário organizar informações em formato de tabela, como usuários, permissões e horários em linhas e colunas, a utilização de matrizes seria uma alternativa possível.

4.1. Entrada e saída

Entrada → dados organizados em linhas e colunas

Processamento → armazenamento e acesso aos dados através de índices de linha e coluna

Saída → exibição organizada das informações em formato tabular